

Explication et Test pour l'oral

TP Final

Nb : Problème d'autorisation entre wsl et Windows (donc certaines parties sur powershell)

Partie 1

Vagrantfile.sh

Fichier d'Infrastructure as Code. Il dit à VirtualBox quelles machines créer pour qu'on n'ait rien à faire à la main .

Test :

- vagrant up (démarrer)
- vagrant reload (redémarrer)
- vagrant status (montrer leur fonctionnement)

.\generate_win.bat (recup l ip)

generate_inventory.sh

Vagrant donne des IPs aléatoires, ce script va interroger les machines en SSH pour récupérer leurs vraies IPs et construire l'inventaire Ansible.

Test :

ouvrir inventory.sh

Ansible va pouvoir installer k3s sur les machines virtuel

Partie 2

Dockerfile

Permet de paqueter l'application Node.js. Nous avons utilisé un multi-stage build pour optimiser la taille comme demandé

test:

```
sudo docker build -t mon-app-v2 .  
docker images  
sudo docker run -d -p 3001:3000 --name test-app-v2 mon-app-v2  
curl http://localhost:3000/
```

Montrer l'image mon app tp

Partie 3

fichiers YAML

Décrivent à K3s comment l'application et la base de données doivent se comporter .

Le PVC (PersistentVolumeClaim) de MySQL : Garantit que les données de la base ne sont pas supprimées si le serveur redémarre.

Le Service ClusterIP de l'API : mpêchEe qu'on puisse pirater l'API depuis l'extérieur du cluster.

Le fichier HPA (Autoscaler) : Permet à K3s de créer jusqu'à 3 copies de l'API si le trafic explose.

test:

```
à taper dans la vm k3s  
sudo kubectl get pods ( montre que l'application et la base tournent )  
sudo kubectl get svc (montre que le type est bien ClusterIP)
```

Partie 4

deploy.sh

Installe Docker, K3s et le monitoring sur toutes les machines

.yml dans .github/workflows/

Dit à Github de lancer le deploy.sh à chaque mise à jour du code

C est la vm “monitoring “ qui interroge github

Ce qui empeche qui github d acceder a notre reseau privé (plus sécuritaire)

test:

Allez sur Github, dans l'onglet "Actions"

Partie 5

install_monitoring.sh

script qui installe la surveillance de la santé ram/cpu

test:

vagrant ssh monitoring -c "hostname -I"

[http://\(Mettre ip\):3000](http://(Mettre ip):3000)

1860 dans le dashboard puis cliquer sur Load

Cliquer sur prometheus et valider

Montrez l'écran Grafana avec les graphiques CPU/RAM

On voit bien mes deux machines : la machine k3s ET la machine monitoring."

Prometheus collecte les données sur les deux machines, et Grafana les affiche sur ce Dashboard dynamique numéro 1860 comme demandé.

Résumé

Partie 1 : 2 fichiers

Vagrantfile (Crée vos deux machines virtuelles).

scripts/generate_inventory.sh (Trouve les IPs et crée l'inventaire Ansible).
inventory.ini qui est créé automatiquement par le script

Partie 2 : 3 fichiers

Dockerfile (La recette de l'image de votre API avec le multi-stage).

package.json (Dépendances du projet Node.js).

index.js (Code de votre API Node.js).

Partie 3 : 1 script + des fichiers YAML

scripts/install_k3s.sh (Le script pour installer le cluster).

Partie 4 : (2 fichiers

.github/workflows/pipeline.yml (La règle Github Actions qui déclenche le tout).

scripts/deploy.sh (Orchestre toute l'installation automatiquement).

Partie 5 : 1 fichier

scripts/install_monitoring.sh (Installe Node Exporter, Prometheus et Grafana).

hyperviseur: Son rôle principal est de permettre à plusieurs systèmes d'exploitation différents de fonctionner en même temps sur une seule machine physique